

# Computer Practical Tools of the Trade

Approximate numerical methods

Paul Connolly • September 2026

## 1 Overview

This is the first practical in the modelling sequence. You will use the command-line workflow introduced in the lecture to download, inspect, edit and run small Python models on the teaching server. At the same time, you will investigate how differential equations are converted into numerical algorithms.

The programs are deliberately short. You do not need to understand every Python or plotting command. Concentrate on the parameters near the top of each file and on the few lines that implement the numerical method.

**By the end of the practical you should be able to:**

- use SSH, Git, nano, Python and SFTP as part of a simple modelling workflow;
- relate a Forward Euler update in Python to the differential equation from which it was derived;
- distinguish the sampling of an exact solution from the time-step of a numerical solution;
- describe numerical error, convergence, numerical diffusion and numerical instability.

## 2 From a differential equation to an algorithm

Consider radioactive decay,

$$\frac{dN}{dt} = -\lambda N, \quad (1)$$

where  $N$  is the number of radioactive atoms,  $t$  is time and  $\lambda$  is the decay constant ( $\text{s}^{-1}$ ). Its exact analytical solution is

$$N(t) = N_0 \exp(-\lambda t). \quad (2)$$

A computer model usually represents time at discrete values separated by a finite time-step  $\Delta t$ . The Forward Euler method approximates the derivative at time level  $n$  as

$$\left. \frac{dN}{dt} \right|_n \approx \frac{N_{n+1} - N_n}{\Delta t}. \quad (3)$$

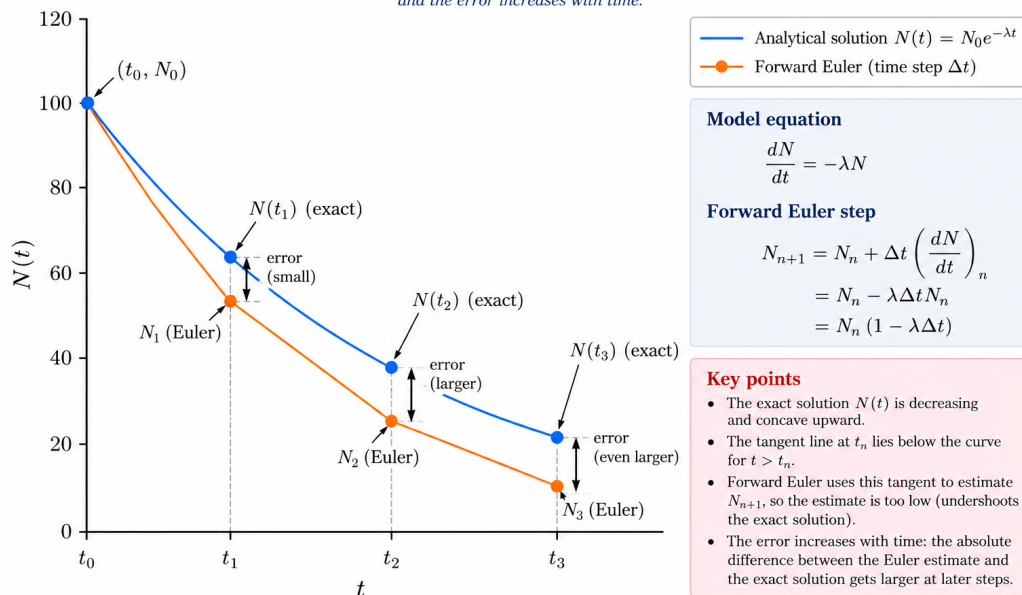
Substituting this into Equation 1 and rearranging gives

$$\boxed{N_{n+1} = N_n - \Delta t \lambda N_n.} \quad (4)$$

Starting from  $N_0$ , Equation 4 is applied repeatedly to advance the solution through time. This is an *approximation*: changing  $\Delta t$  generally changes the numerical solution. As  $\Delta t$  is reduced, a well-behaved numerical method should approach the exact solution.

### Forward Euler method for exponential decay

The Euler estimate uses the gradient at the start of each step, so it lies below the exact solution, and the error increases with time.



**Figure 1:** Forward Euler applied to a decay problem. At each time level, the governing equation gives the gradient; that gradient is used to advance the numerical solution by one finite time-step  $\Delta t$ .

### 3 Log in and download the code

Use the SSH and SFTP workflow introduced in the lecture. The same commands can be entered in Terminal on macOS/ChromeOS or PowerShell/CMD on Windows. Replace YOUR\_USERNAME and SERVER\_ADDRESS with the details provided for the course.

In your SSH window:

```
ssh -Y YOUR_USERNAME@SERVER_ADDRESS
```

Open a second terminal window for file transfer:

```
sftp YOUR_USERNAME@SERVER_ADDRESS
```

Your password will not be displayed while you type it.

In the SSH window, download the practical repository and enter it:

```
git clone https://github.com/EnvModelling/approximate-methods-practical
cd approximate-methods-practical
pwd
ls
```

pwd shows where you are on the server; ls lists the files in the current directory. You should see three Python scripts: approximate\_methods01.py, approximate\_methods02.py and approximate\_methods03.py.

### 4 Inspecting and running a Python model

Open the first script with line numbers displayed:

```
nano -l approximate_methods01.py
```

In nano, Ctrl+W searches for text. Try searching for PARAMETERS TO CHANGE and then for FORWARD EULER STEP. In the current repository, the parameters in this first script are around lines 14–18 and the Forward Euler update is around line 38. Compare that line with Equation 4.

**Line numbers are approximate.** They are useful for finding your way around, but they may move slightly if the code is updated. Searching for the parameter name or the labelled code section is more reliable.

Save and exit using Ctrl-X, then Y, followed by Enter. If you have only been looking and have made no changes, you can simply exit.

Run the script with:

```
python3 approximate_methods01.py
```

The script prints the full path of the file it creates. For `approximate_methods01.py`, the output file is always called `approximate_methods01.png` and is written to `/tmp/YOUR_USERNAME/`. From the SSH shell, list your temporary output directory with:

```
ls /tmp/$USER/
```

Here `$USER` is expanded automatically by the Linux shell to your username.

**What “download as” means.** In SFTP, the command

```
get REMOTE_FILE LOCAL_FILENAME
```

copies a file from the server to the computer on which your SFTP window is running. The first name is the file on the server; the optional second name is what you want to call the downloaded copy on your own computer.

For example, after the default run of script 01, type in the **SFTP window**:

```
get /tmp/YOUR_USERNAME/approximate_methods01.png decay_default.png
```

This downloads the server file `approximate_methods01.png` but saves your local copy as `decay_default.png`. Giving each downloaded result a meaningful local name prevents later runs from overwriting your earlier results.

**Returning to the supplied defaults.** During the practical you will deliberately edit the scripts. When instructed, the following command restores tracked files in the repository to the downloaded version:

```
git reset --hard
```

This **discards edits you have made inside the repository**, so use it only when you intend to return to the supplied defaults. It does not remove files already written to `/tmp/$USER/`.

## 5 Parameters used in the practical

Table 1 gives a quick guide to where the important parts of each script are located and to the output file each script creates. These line numbers refer to the current teaching repository and are approximate.

**Table 1:** Quick guide to the three Python scripts.

| Script | Parameters  | Numerical step | Output written to <code>/tmp/\$USER/</code> |
|--------|-------------|----------------|---------------------------------------------|
| 01     | lines 14–18 | line 38        | <code>approximate_methods01.png</code>      |
| 02     | lines 14–19 | lines 44–45    | <code>approximate_methods02.png</code>      |
| 03     | lines 15–21 | line 60        | <code>animation.gif</code>                  |

Table 2 summarises the parameters you will change. Notice the distinction between `dt_plot` and a numerical time-step. The exact analytical formula itself does not have a time-step; `dt_plot` only determines how often we evaluate it for plotting.

**Table 2:** Key parameters in the three scripts.

| Script | Parameter | Default              | Meaning                                                           |
|--------|-----------|----------------------|-------------------------------------------------------------------|
| 01     | dt_plot   | 1 s                  | spacing used to sample the exact decay solution for plotting      |
| 01     | dt_euler  | 10 s                 | Forward Euler numerical time-step                                 |
| 02     | dt_plot   | 1 s                  | spacing used to sample the exact oscillator solution for plotting |
| 02     | dt_euler  | 0.5 s                | Forward Euler numerical time-step                                 |
| 03     | dx        | 100 m                | spatial grid spacing                                              |
| 03     | dt        | 2.5 s                | upwind numerical time-step                                        |
| 03     | v         | 20 m s <sup>-1</sup> | transport velocity                                                |

## 6 Experiments

**Keep a practical record.** Save each figure or animation and make brief notes as you go. Record the parameter values used for each run and what changed in the output. You may be asked to interpret these experiments in the assessment.

### 6.1 Radioactive decay: time-step and convergence

The parameters you need are near the top of `approximate_methods01.py` (around lines 14–18). The Forward Euler update itself is around line 38. The script always writes `/tmp/$USER/approximate_methods01.png`; after each run you will download that same remote file using a different local filename.

First run `approximate_methods01.py` using the defaults. In the SFTP window, download the output as:

```
get /tmp/YOUR_USERNAME/approximate_methods01.png decay_default.png
```

**Exercise.** Using the default values  $N_0 = 10^{20}$ ,  $\lambda = 1/2000 \text{ s}^{-1}$  and  $\Delta t = 10 \text{ s}$ , use Equation 4 to calculate  $N_1$  by hand. Find the line in the Python script that performs exactly this calculation.

Now carry out the runs in Table 3. Change only the indicated parameter before each run. Run the script again, then use the same SFTP get command pattern to download `approximate_methods01.png` using the suggested local filename in the final column.

**Table 3:** Radioactive-decay experiments.

| Run         | dt_plot (s) | dt_euler (s) | Suggested local filename |
|-------------|-------------|--------------|--------------------------|
| A (default) | 1           | 10           | decay_default.png        |
| B           | 1000        | 10           | decay_dtplot1000.png     |
| C           | 1           | 1000         | decay_dteuler1000.png    |
| D           | 1           | 10000        | decay_dteuler10000.png   |

**Exercise.** Compare runs A and B. Has the analytical solution changed, or have you simply sampled the same exact function at fewer times? What in the plot tells you this?

**Exercise.** Compare runs A, C and D. How does increasing the Forward Euler time-step affect the numerical solution? Run D can even produce a negative value of  $N$ . Why is that physically impossible, and what does it tell you about the numerical approximation?

**Exercise.** In general, what should happen to the Forward Euler solution as `dt_euler` is reduced? This behaviour is called *convergence*.

Before continuing, restore the supplied files:

```
git reset --hard
```

## 6.2 Second-order ODE: a harmonic oscillator

The second script solves

$$\frac{d^2y}{dt^2} = -\omega^2 y. \quad (5)$$

Introducing velocity  $v = dy/dt$  converts this into two coupled first-order equations,

$$\frac{dy}{dt} = v, \quad \frac{dv}{dt} = -\omega^2 y. \quad (6)$$

Open `approximate_methods02.py`. Its editable parameters are around lines 14–19; search for `FORWARD EULER STEPS` to find the two numerical updates around lines 44–45. Identify the two Python statements corresponding to these equations.

This script always writes `/tmp/$USER/approximate_methods02.png`. Run the default case (`dt_euler=0.5`) and, in SFTP, download it as:

```
get /tmp/YOUR_USERNAME/approximate_methods02.png oscillator_dt0p5.png
```

Then change only

```
dt_euler = 0.05
```

Run the script again and download the new output as:

```
get /tmp/YOUR_USERNAME/approximate_methods02.png oscillator_dt0p05.png
```

**Exercise.** The exact oscillator has constant amplitude. What happens to the amplitude of the Forward Euler solution with `dt_euler=0.5`? What does this tell you about the numerical method?

**Exercise.** How does reducing the time-step to 0.05 s change the numerical solution? Relate your answer to convergence.

Restore the defaults before the final experiment:

```
git reset --hard
```

## 6.3 Transport equation: numerical diffusion and stability

The third script previews a partial differential equation that we will study later. It transports a triangular plume along the  $x$ -direction using a first-order upwind scheme and compares the numerical result with the exact translated shape.

The main parameters are around lines 15–21 of `approximate_methods03.py`; the first-order upwind update is around line 60. This script creates an animated GIF called `animation.gif` in `/tmp/$USER/`.

Run:

```
python3 approximate_methods03.py
```

The script prints the Courant–Friedrichs–Lewy (CFL) number. In the SFTP window, download the resulting `animation.gif` as:

```
get /tmp/YOUR_USERNAME/animation.gif transport_cfl0p5.gif
```

Open the GIF in a web browser if your normal image viewer does not animate it.

For this simple problem,

$$C = \frac{|v|\Delta t}{\Delta x}. \quad (7)$$

With the default values,  $C = 20 \times 2.5/100 = 0.5$ . Physically, this means the plume moves half a grid cell during one numerical time-step.

**Exercise.** Compare the upwind and exact solutions as time progresses. The governing equation contains no physical diffusion, yet the numerical plume becomes broader and flatter. This artificial smoothing is called *numerical diffusion*. Describe what you see.

Now change  $\Delta t$  to 5 s (around line 16). Before running the model, calculate the CFL number and predict whether the plume moves less than, exactly, or more than one grid cell per time-step. Run the model, then download the new `animation.gif` as:

```
get /tmp/YOUR_USERNAME/animation.gif transport_cfl1p0.gif
```

**Exercise.** What happens to the numerical diffusion when  $C = 1$ ? Why might this particular value be special for this simple constant-velocity, one-dimensional upwind calculation?

Finally set  $\Delta t = 6$  s.

**Prediction.** Calculate  $C$  before running the model. Based on the rule that this upwind scheme requires  $C \leq 1$  for stability, what do you expect to happen?

Run the model and download the output as:

```
get /tmp/YOUR_USERNAME/animation.gif transport_cfl1p2.gif
```

**Exercise.** What happens to the numerical solution? This is *numerical instability*: the growing oscillations are produced by the numerical method, not by the environmental process being modelled.

## 7 What you should take away

A numerical model is not simply “the equation on a computer”. The continuous equations must be represented on discrete grids and advanced using numerical algorithms. The choice of numerical method and time-step can therefore affect accuracy, introduce artificial behaviour such as numerical diffusion, or even make a solution unstable. These ideas will recur throughout the course.

You should also now have practised the basic workflow that we will use repeatedly: **log in** → **obtain code** → **inspect/edit** → **run** → **retrieve output** → **interpret the result**.